

SWITCHBUTTON

1.0.1

Generated by Doxygen 1.10.0

1 SWITCHBUTTON	1
1 SWITCHBUTTON	1
2 Class Index	2
2.1 Class List	2
3 File Index	2
3.1 File List	2
4 Class Documentation	3
4.1 SWITCHBUTTON Class Reference	3
4.1.1 Detailed Description	3
4.1.2 Member Enumeration Documentation	3
4.1.3 Constructor & Destructor Documentation	4
4.1.4 Member Function Documentation	5
5 File Documentation	8
5.1 buttonStates.ino	8
5.2 pinChangeInterrupt.ino	8
5.3 simple.ino	9
5.4 sleepmode.ino	10
5.5 SWITCHBUTTON.h File Reference	10
5.5.1 Detailed Description	11
5.5.2 Macro Definition Documentation	12
5.6 SWITCHBUTTON.h	13
Index	15

1 SWITCHBUTTON

An Arduino Uno/Nano library to use momentary switch buttons with debouncing, short and long press detection, supports sleep modes and works without blocking or delay()



Figure 1 Switch buttons

Examples how to use the library

- [examples/simple/simple.ino](#)
- [examples/buttonStates/buttonStates.ino](#)
- [examples/sleepmode/sleepmode.ino](#)
- [examples/pinChangeInterrupt/pinChangeInterrupt.ino](#)

2 Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

[SWITCHBUTTON](#)

3

3 File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

buttonStates.ino	8
pinChangeInterrupt.ino	8
simple.ino	9
sleepmode.ino	10
SWITCHBUTTON.h	10

4 Class Documentation

4.1 SWITCHBUTTON Class Reference

```
#include <SWITCHBUTTON.h>
```

Public Types

- enum [buttonLogicalStates](#) {
 [IDLE](#) , [SHORTPRESSED](#) , [LONGPRESSED](#) , [LONGPRESSEDRELEASED](#) ,
 [INPROGRESS](#) , [MISSED](#) , [DEBOUNCING](#) }
- enum [buttonPhysicalStates](#) { [RELEASED](#) , [PRESSED](#) , [UNKNOWN](#) }

Public Member Functions

- [SWITCHBUTTON](#) (byte sw_pin, bool inputPulledUp=true)
 Constructor of a the momentary switch object.
- byte [checkButton](#) ()
 Returns the current logical state from stored button state.
- byte [getButton](#) ()
 Get current logical state for momentary switch.
- byte [getState](#) ()
 Get last stored physical button state for the momentary switch.
- bool [readyForSleep](#) ()
 Checks, if it save to go to sleep deeper than SLEEP_MODE_IDLE.
- void [setDebounceTimeMS](#) (unsigned int debounceTimeMS)
 Change debounce time for a rising or falling signal edge.
- void [setState](#) (byte state)
 Store physical button state for the momentary switch.

4.1.1 Detailed Description

Class for a momentary switch button.

Definition at line 34 of file [SWITCHBUTTON.h](#).

4.1.2 Member Enumeration Documentation

buttonLogicalStates

```
enum SWITCHBUTTON::buttonLogicalStates
```

Button logical states

Enumerator

IDLE	Button is not pressed and is idle
SHORTPRESSED	Button was short pressed
LONGPRESSED	Button was long pressed
LONGPRESSEDRELEASED	Button was released after a long press
INPROGRESS	Button press is in progress
MISSED	Incomplete long press was detected
DEBOUNCING	Button is blocked for debouncing

Definition at line 37 of file [SWITCHBUTTON.h](#).

```

00038     {
00039         IDLE,
00040         SHORTPRESSED,
00041         LONGPRESSED,
00042         LONGPRESSEDRELEASED,
00043         INPROGRESS,
00044         MISSED,
00045         DEBOUNCING
00046     };

```

buttonPhysicalStates

```
enum SWITCHBUTTON::buttonPhysicalStates
```

Button physical states

Enumerator

RELEASED	Button not pressed
PRESSED	Button pressed
UNKNOWN	Button never checked or set

Definition at line 49 of file [SWITCHBUTTON.h](#).

```

00050     {
00051         RELEASED,
00052         PRESSED,
00053         UNKNOWN
00054     };

```

4.1.3 Constructor & Destructor Documentation**SWITCHBUTTON()**

```

SWITCHBUTTON::SWITCHBUTTON (
    byte sw_pin,
    bool inputPulledUp = true ) [inline]

```

Constructor of a the momentary switch object.

Parameters

in	<i>sw_pin</i>	Digital input pin connected to the momentary switch
in	<i>inputPulledUp</i>	Set to true (=default), if momentary switch is pulled up by a resistor or <code>pinMode(,INPUT_PULLUP)</code> otherwise set parameter to false

Definition at line 62 of file SWITCHBUTTON.h.

```
00063     {
00064         m_sw_pin = sw_pin; // Digital input pin for the button
00065         m_inputPulledUp = inputPulledUp; // Has button a pullup resistor (LOW=pressed,HIGH=released)?
00066         m_debounceTimeMS = DEBOUNCEMS;
00067         mv_state = UNKNOWN;
00068         mv_lastState = UNKNOWN;
00069         mv_lastLongPressedMS = 0;
00070         mv_lastButtonStartMS = 0;
00071         mv_lastButtonChangeMS=0;
00072         mv_waitingRelease = false;
00073         mv_pendingLongPressed = false;
00074     }
```

4.1.4 Member Function Documentation

checkButton()

```
byte SWITCHBUTTON::checkButton ( ) [inline]
```

Returns the current logical state from stored button state.

If you do not use interrupts, you have to start [setState\(\)](#) and [checkButton\(\)](#) or a function using these (for example [getButton\(\)](#)) very frequently in your loop to prevent missing button presses

Return values

SWITCHBUTTON::IDLE	Button is not pressed and is idle
SWITCHBUTTON::SHORTPRESSED	Button was short pressed
SWITCHBUTTON::LONGPRESSED	Button was long pressed
SWITCHBUTTON::LONGPRESSEDRELEASED	Button was released after a long pressed
SWITCHBUTTON::INPROGRESS	Button press is in progress
SWITCHBUTTON::MISSED	Incomplete long press was detected
SWITCHBUTTON::DEBOUNCING	Button is blocked for debouncing

Definition at line 90 of file SWITCHBUTTON.h.

```
00091     {
00092         unsigned long currentMillis = millis();
00093         // When first check or after debounce time
00094         if ((mv_lastState == UNKNOWN)
00095             || (currentMillis-mv_lastButtonChangeMS > m_debounceTimeMS)) {
00096             mv_lastButtonChangeMS = currentMillis - m_debounceTimeMS - 1; // Prevent overrun
00097             if ((mv_state==RELEASED) && (mv_lastState == UNKNOWN)) { // Init
00098                 mv_lastState = mv_state;
00099                 return IDLE;
00100             }
00101             if (mv_state != mv_lastState) { // Button state has changed
00102                 if (mv_state == PRESSED) { // Rising edge
00103                     mv_waitingRelease = true;
00104                     mv_lastButtonStartMS = currentMillis;
00105                 }
00106                 mv_lastButtonChangeMS = currentMillis;
00107                 mv_lastState = mv_state;
00108             }
00109             if (mv_state == PRESSED) { // Button is pressed
00110                 if (mv_waitingRelease) {
00111                     if ((currentMillis - mv_lastButtonStartMS > LONGPRESSEDMS)
00112                         && (currentMillis-mv_lastLongPressedMS > LONGPRESSEDDEADTIMES)) {
00113                         mv_lastButtonStartMS = currentMillis - LONGPRESSEDMS - 1; // Prevent overrun
00114                         mv_lastLongPressedMS = currentMillis;
00115                         mv_pendingLongPressed = true;
00116                         return LONGPRESSED;
00117                     } else return INPROGRESS;
00118                 } else return INPROGRESS;
00119             } else { // Button is released
00120                 if (mv_waitingRelease) {
00121                     mv_waitingRelease = false;
00122                     if (currentMillis - mv_lastButtonStartMS <= LONGPRESSEDMS){
```

```

00123         return SHORTPRESSED;
00124     } else {
00125         if (mv_pendingLongPressed) {
00126             mv_pendingLongPressed = false;
00127             return LONGPRESSEDRELEASED;
00128         }
00129         return MISSED; // Too long gap between rising edge and button release
00130     }
00131     } else return IDLE;
00132 }
00133 } else { // In debounce time
00134     return DEBOUNCING;
00135 }
00136 }

```

getButton()

```
byte SWITCHBUTTON::getButton ( ) [inline]
```

Get current logical state for momentary switch.

Reads physical button state with DigitalRead() and checks current logical state by calling [checkButton\(\)](#)

Return values

SWITCHBUTTON::IDLE	Button is not pressed and is idle
SWITCHBUTTON::SHORTPRESSED	Button was short pressed
SWITCHBUTTON::LONGPRESSED	Button was long pressed
SWITCHBUTTON::LONGPRESSEDRELEASED	Button was released after a long pressed
SWITCHBUTTON::INPROGRESS	Button press is in progress
SWITCHBUTTON::MISSED	Incomplete long press was detected
SWITCHBUTTON::DEBOUNCING	Button is blocked for debouncing

Definition at line 151 of file [SWITCHBUTTON.h](#).

```

00152 {
00153     if (m_inputPulledUp) {
00154         setState( (digitalRead(m_sw_pin)==LOW) ? PRESSED:RELEASED);
00155     } else {
00156         setState( (digitalRead(m_sw_pin)==HIGH) ? PRESSED:RELEASED);
00157     }
00158     return checkButton();
00159 }

```

getState()

```
byte SWITCHBUTTON::getState ( ) [inline]
```

Get last stored physical button state for the momentary switch.

Return values

SWITCHBUTTON::IDLE	Button was not pressed and is idle
SWITCHBUTTON::PRESSED	Button was pressed
SWITCHBUTTON::UNKNOWN	Button was never checked or set

Definition at line 168 of file [SWITCHBUTTON.h](#).

```
00169 {
```

```
00170         return mv_state;
00171     }
```

readyForSleep()

```
bool SWITCHBUTTON::readyForSleep ( ) [inline]
```

Checks, if it save to go to sleep deeper than SLEEP_MODE_IDLE.

Returns true, if device has no pending button press. Sleep mode SLEEP_MODE_IDLE is always possible but deeper sleep modes, for example SLEEP_MODE_PWR_SAVE, are only save after [readyForSleep\(\)](#) returns true

Return values

<i>true</i>	Yes, it is save to go to sleep deeper then SLEEP_MODE_IDLE
<i>false</i>	No, only SLEEP_MODE_IDLE is possible

Definition at line 184 of file [SWITCHBUTTON.h](#).

```
00185     {
00186         return (checkButton() == IDLE);
00187     }
```

setDebounceTimeMS()

```
void SWITCHBUTTON::setDebounceTimeMS (
    unsigned int debounceTimeMS ) [inline]
```

Change debounce time for a rising or falling signal edge.

Parameters

in	<i>debounceTimeMS</i>	Debounce time in milliseconds
----	-----------------------	-------------------------------

Definition at line 194 of file [SWITCHBUTTON.h](#).

```
00195     {
00196         m_debounceTimeMS = debounceTimeMS;
00197     }
```

setState()

```
void SWITCHBUTTON::setState (
    byte state ) [inline]
```

Store physical button state for the momentary switch.

Parameters

in	<i>state</i>	Button state: SWITCHBUTTON::PRESSED (=pressed) or SWITCHBUTTON::RELEASED (=released).
----	--------------	---

Definition at line 204 of file [SWITCHBUTTON.h](#).


```

00205     {
00206         if (state == PRESSED ) mv_state = PRESSED;
00207         if (state == RELEASED ) mv_state = RELEASED;
00208     }

```

The documentation for this class was generated from the following file:

- [SWITCHBUTTON.h](#)

5 File Documentation

5.1 buttonStates.ino

```

00001 /*
00002  * Example to get all button states
00003  */
00004
00005 #include <SWITCHBUTTON.h>
00006
00007 // Button pin
00008 #define SW_PIN 6
00009 // Button object
00010 SWITCHBUTTON switchButton(SW_PIN);
00011
00012 void setup() {
00013     Serial.begin(9600);
00014     // If you have no external pullup resistor, set pin to INPUT_PULLUP
00015     pinMode(SW_PIN, INPUT_PULLUP);
00016 }
00017
00018 void loop() {
00019     static byte lastButtonState = SWITCHBUTTON::UNKNOWN;
00020     byte buttonState;
00021
00022     // Start getButton() frequently in your loop to avoid missing button presses
00023     buttonState = switchButton.getButton();
00024     if (buttonState != lastButtonState) {
00025         lastButtonState = buttonState;
00026         switch (buttonState) {
00027             case SWITCHBUTTON::SHORTPRESSED:
00028                 Serial.println("Pressed");
00029                 break;
00030             case SWITCHBUTTON::LONGPRESSED:
00031                 Serial.println("Long pressed");
00032                 break;
00033             case SWITCHBUTTON::MISSED:
00034                 Serial.println("Missed");
00035                 break;
00036             case SWITCHBUTTON::INPROGRESS:
00037                 Serial.println("In progress");
00038                 break;
00039             case SWITCHBUTTON::IDLE:
00040                 Serial.println("Idle");
00041                 break;
00042             case SWITCHBUTTON::DEBOUNCING:
00043                 Serial.println("Debouncing");
00044                 break;
00045             case SWITCHBUTTON::LONGPRESSEDRELEASED:
00046                 Serial.println("Long pressed released");
00047                 break;
00048         }
00049     }
00050 }

```

5.2 pinChangeInterrupt.ino

```

00001 /*
00002  * Example to get the final button press state with a "bad", blocking loop
00003  * (for example a delay or long running function) by using pin change interrupts
00004  */
00005
00006 #include "switchButton.h"
00007
00008 // Button pin
00009 #define SW_PIN 6

```

```

00010 // Button object
00011 SWITCHBUTTON switchButton(SW_PIN);
00012
00013 // Button states
00014 volatile bool v_shortPressed = false;
00015 volatile bool v_longPressed = false;
00016
00017 // Enable pin change interrupt
00018 void pciSetup(byte pin) {
00019     *digitalPinToPCMSK(pin) |= bit (digitalPinToPCMSKbit(pin)); // enable pin
00020     PCIFR |= bit (digitalPinToPCICRbit(pin)); // clear any outstanding interrupt
00021     PCICR |= bit (digitalPinToPCICRbit(pin)); // enable interrupt for the group
00022 }
00023
00024 // Update button states (will be used in ISR and loop)
00025 void updateSwitchStates() {
00026     switch (switchButton.getButton()) {
00027         case SWITCHBUTTON::SHORTPRESSED:
00028             v_shortPressed = true;
00029             break;
00030         case SWITCHBUTTON::LONGPRESSED:
00031             v_longPressed = true;
00032             break;
00033     }
00034 }
00035
00036 // ISR to handle pin change interrupt for D0 to D7 here
00037 ISR (PCINT2_vect) {
00038     // Update button states from momentary switch
00039     updateSwitchStates();
00040 }
00041
00042 void setup() {
00043     Serial.begin(9600);
00044
00045     // If you have no external pullup resistor, set pin to INPUT_PULLUP
00046     pinMode(SW_PIN, INPUT_PULLUP);
00047
00048     // Set pin change interrupt for momentary switch
00049     pciSetup(SW_PIN);
00050 }
00051
00052 void loop() {
00053     // For local copies of button states
00054     bool shortPressed = false;
00055     bool longPressed = false;
00056
00057     // Example for a "bad", blocking function in loop
00058     delay(2000);
00059     // Blocking longer than LONGPRESSEDMS or LONGPRESSEDDEADTIMEMS will delay the long pressed detection
00060
00061     // Update button states from momentary switch, make an "interrupt save" copy and reset the states
00062     cli();
00063     updateSwitchStates();
00064     shortPressed = v_shortPressed;
00065     longPressed = v_longPressed;
00066     v_shortPressed = false;
00067     v_longPressed = false;
00068     sei();
00069
00070     // Show button state
00071
00072     if (shortPressed) { // Short pressed?
00073         Serial.println("Short pressed");
00074     }
00075
00076     if (longPressed) { // Long pressed?
00077         Serial.println("Long pressed");
00078     }
00079 }

```

5.3 simple.ino

```

00001 /*
00002  * Example to get the final button press state
00003  */
00004
00005 #include <SWITCHBUTTON.h>
00006
00007 // Button pin
00008 #define SW_PIN 6
00009 // Button object
00010 SWITCHBUTTON switchButton(SW_PIN);
00011

```

```

00012 void setup() {
00013     Serial.begin(9600);
00014     // If you have no external pullup resistor, set pin to INPUT_PULLUP
00015     pinMode(SW_PIN, INPUT_PULLUP);
00016 }
00017
00018 void loop() {
00019     // Start getButton() frequently in your loop to avoid missing button presses
00020     switch (switchButton.getButton()) {
00021         case SWITCHBUTTON::SHORTPRESSED:
00022             Serial.println("Pressed");
00023             break;
00024         case SWITCHBUTTON::LONGPRESSED:
00025             Serial.println("Long pressed");
00026             break;
00027     }
00028 }

```

5.4 sleepmode.ino

```

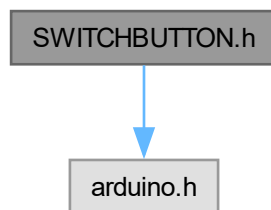
00001 /*
00002  * Example to get the final button press state while using sleep modes
00003  */
00004
00005 #include <avr/sleep.h>
00006 #include <SWITCHBUTTON.h>
00007
00008 // Button pin
00009 #define SW_PIN 6
00010 SWITCHBUTTON switchButton(SW_PIN);
00011
00012 // Enable pin change interrupt
00013 void pciSetup(byte pin) {
00014     *digitalPinToPCMSK(pin) |= bit (digitalPinToPCMSKbit(pin)); // enable pin
00015     PCIFR |= bit (digitalPinToPCICRbit(pin)); // clear any outstanding interrupt
00016     PCICR |= bit (digitalPinToPCICRbit(pin)); // enable interrupt for the group
00017 }
00018
00019 // ISR to handle pin change interrupt for D0 to D7 here
00020 ISR (PCINT2_vect) {
00021     // Empty, only as a wakeup trigger
00022 }
00023
00024 void setup() {
00025     Serial.begin(9600);
00026
00027     // If you have no external pullup resistor for the button, set pin to INPUT_PULLUP
00028     pinMode(SW_PIN, INPUT_PULLUP);
00029
00030     // Set pin change interrupt for momentary switch
00031     pciSetup(SW_PIN);
00032 }
00033
00034 void loop() {
00035     // Default sleep mode (SWITCHBUTTON library allows everytime SLEEP_MODE_IDLE)
00036     byte selectedSleepMode = SLEEP_MODE_IDLE;
00037
00038     // Start getButton() frequently in your loop to avoid missing button presses
00039     switch (switchButton.getButton()) {
00040         case SWITCHBUTTON::SHORTPRESSED:
00041             Serial.println("Short pressed");
00042             break;
00043         case SWITCHBUTTON::LONGPRESSED:
00044             Serial.println("Long pressed");
00045             break;
00046         case SWITCHBUTTON::IDLE:
00047             // Set deeper sleep mode, because now we need no millis()&Co
00048             selectedSleepMode = SLEEP_MODE_PWR_DOWN;
00049             break;
00050     }
00051     Serial.flush();
00052     set_sleep_mode(selectedSleepMode);
00053     sleep_mode();
00054 }
00055

```

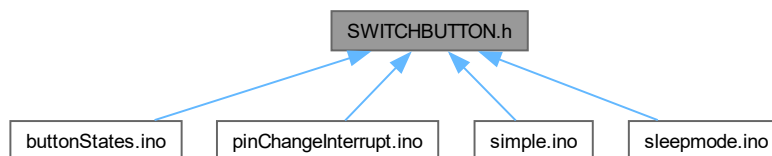
5.5 SWITCHBUTTON.h File Reference

```
#include <arduino.h>
```

Include dependency graph for SWITCHBUTTON.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [SWITCHBUTTON](#)

Macros

- `#define` [SWITCHBUTTON_VERSION](#) "1.0.1"
- `#define` [LONGPRESSEDMS](#) 1000
- `#define` [DEBOUNCEMS](#) 100
- `#define` [LONGPRESSEDDEADTIMEMS](#) 500

5.5.1 Detailed Description

Class: [SWITCHBUTTON](#)

Arduino Library for a momentary switch button.

This library

- detects and differentiates between short and long button presses
- supports debouncing
- works nonblocking and without delay()
- supports sleep modes (see example "sleepmode")

Home: <https://github.com/codingABI/SWITCHBUTTON>

Author

codingABI <https://github.com/codingABI/>

Copyright

CC0

Version

1.0.1

Definition in file [SWITCHBUTTON.h](#).

5.5.2 Macro Definition Documentation

DEBOUNCEMS

```
#define DEBOUNCEMS 100
```

Default debounce time in milliseconds after a rising or falling signal edge

Definition at line 29 of file [SWITCHBUTTON.h](#).

LONGPRESSEDDEADTIMEMS

```
#define LONGPRESSEDDEADTIMEMS 500
```

Splits a sustained long press in individuals long presses after these milliseconds

Definition at line 31 of file [SWITCHBUTTON.h](#).

LONGPRESSEDMS

```
#define LONGPRESSEDMS 1000
```

Time duration in milliseconds to detect a button long press

Definition at line 27 of file [SWITCHBUTTON.h](#).

SWITCHBUTTON_VERSION

```
#define SWITCHBUTTON_VERSION "1.0.1"
```

Library version

Definition at line 22 of file [SWITCHBUTTON.h](#).

5.6 SWITCHBUTTON.h

[Go to the documentation of this file.](#)

```

00001
00019 #pragma once
00020
00022 #define SWITCHBUTTON_VERSION "1.0.1"
00023
00024 #include <arduino.h>
00025
00027 #define LONGPRESSEDMS 1000
00029 #define DEBOUNCEMS 100
00031 #define LONGPRESSEDDEADTIMEMS 500
00032
00034 class SWITCHBUTTON {
00035 public:
00037     enum buttonLogicalStates
00038     {
00039         IDLE,
00040         SHORTPRESSED,
00041         LONGPRESSED,
00042         LONGPRESSEDRELEASED,
00043         INPROGRESS,
00044         MISSED,
00045         DEBOUNCING
00046     };
00047
00049     enum buttonPhysicalStates
00050     {
00051         RELEASED,
00052         PRESSED,
00053         UNKNOWN
00054     };
00055
00062     SWITCHBUTTON(byte sw_pin, bool inputPulledUp=true)
00063     {
00064         m_sw_pin = sw_pin; // Digital input pin for the button
00065         m_inputPulledUp = inputPulledUp; // Has button a pullup resistor (LOW=pressed,HIGH=released)?
00066         m_debounceTimeMS = DEBOUNCEMS;
00067         mv_state = UNKNOWN;
00068         mv_lastState = UNKNOWN;
00069         mv_lastLongPressedMS = 0;
00070         mv_lastButtonStartMS = 0;
00071         mv_lastButtonChangeMS=0;
00072         mv_waitingRelease = false;
00073         mv_pendingLongPressed = false;
00074     }
00075
00090     byte checkButton()
00091     {
00092         unsigned long currentMillis = millis();
00093         // When first check or after debounce time
00094         if ((mv_lastState == UNKNOWN)
00095             || (currentMillis-mv_lastButtonChangeMS > m_debounceTimeMS)) {
00096             mv_lastButtonChangeMS = currentMillis - m_debounceTimeMS - 1; // Prevent overrun
00097             if ((mv_state==RELEASED) && (mv_lastState == UNKNOWN)) { // Init
00098                 mv_lastState = mv_state;
00099                 return IDLE;
00100             }
00101             if (mv_state != mv_lastState) { // Button state has changed
00102                 if (mv_state == PRESSED) { // Rising edge
00103                     mv_waitingRelease = true;
00104                     mv_lastButtonStartMS = currentMillis;
00105                 }
00106                 mv_lastButtonChangeMS = currentMillis;
00107                 mv_lastState = mv_state;
00108             }
00109             if (mv_state == PRESSED) { // Button is pressed
00110                 if (mv_waitingRelease) {
00111                     if ((currentMillis - mv_lastButtonStartMS > LONGPRESSEDMS)
00112                         && (currentMillis-mv_lastLongPressedMS > LONGPRESSEDDEADTIMEMS)) {
00113                         mv_lastButtonStartMS = currentMillis - LONGPRESSEDMS - 1; // Prevent overrun
00114                         mv_lastLongPressedMS = currentMillis;
00115                         mv_pendingLongPressed = true;
00116                         return LONGPRESSED;
00117                     } else return INPROGRESS;
00118                 } else return INPROGRESS;
00119             } else { // Button is released
00120                 if (mv_waitingRelease) {
00121                     mv_waitingRelease = false;
00122                     if (currentMillis - mv_lastButtonStartMS <= LONGPRESSEDMS) {
00123                         return SHORTPRESSED;
00124                     } else {
00125                         if (mv_pendingLongPressed) {
00126                             mv_pendingLongPressed = false;
00127                             return LONGPRESSEDRELEASED;

```

```
00128         }
00129         return MISSED; // Too long gap between rising edge and button release
00130     }
00131     } else return IDLE;
00132     }
00133     } else { // In debounce time
00134         return DEBOUNCING;
00135     }
00136 }
00137
00151 byte getButton()
00152 {
00153     if (m_inputPulledUp) {
00154         setState((digitalRead(m_sw_pin)==LOW) ? PRESSED:RELEASED);
00155     } else {
00156         setState((digitalRead(m_sw_pin)==HIGH) ? PRESSED:RELEASED);
00157     }
00158     return checkButton();
00159 }
00160
00168 byte getState()
00169 {
00170     return mv_state;
00171 }
00172
00184 bool readyForSleep()
00185 {
00186     return (checkButton() == IDLE);
00187 }
00188
00194 void setDebounceTimeMS(unsigned int debounceTimeMS)
00195 {
00196     m_debounceTimeMS = debounceTimeMS;
00197 }
00198
00204 void setState(byte state)
00205 {
00206     if (state == PRESSED ) mv_state = PRESSED;
00207     if (state == RELEASED ) mv_state = RELEASED;
00208 }
00209
00210 private:
00211     byte m_sw_pin;
00212     bool m_inputPulledUp;
00213     unsigned int m_debounceTimeMS;
00214     volatile unsigned long mv_lastLongPressedMS;
00215     volatile unsigned long mv_lastButtonStartMS;
00216     volatile unsigned long mv_lastButtonChangeMS;
00217     volatile byte mv_state;
00218     volatile byte mv_lastState;
00219     volatile bool mv_waitingRelease;
00220     volatile bool mv_pendingLongPressed;
00221 };
```

Index

- buttonLogicalStates
 - SWITCHBUTTON, [3](#)
- buttonPhysicalStates
 - SWITCHBUTTON, [4](#)
- buttonStates.ino, [8](#)
- checkButton
 - SWITCHBUTTON, [5](#)
- DEBOUNCEMS
 - SWITCHBUTTON.h, [12](#)
- DEBOUNCING
 - SWITCHBUTTON, [4](#)
- getButton
 - SWITCHBUTTON, [6](#)
- getState
 - SWITCHBUTTON, [6](#)
- IDLE
 - SWITCHBUTTON, [4](#)
- INPROGRESS
 - SWITCHBUTTON, [4](#)
- LONGPRESSED
 - SWITCHBUTTON, [4](#)
- LONGPRESSEDDEADTIMEMS
 - SWITCHBUTTON.h, [12](#)
- LONGPRESSEDMS
 - SWITCHBUTTON.h, [12](#)
- LONGPRESSEDRELEASED
 - SWITCHBUTTON, [4](#)
- MISSED
 - SWITCHBUTTON, [4](#)
- pinChangeInterrupt.ino, [8](#)
- PRESSED
 - SWITCHBUTTON, [4](#)
- readyForSleep
 - SWITCHBUTTON, [7](#)
- RELEASED
 - SWITCHBUTTON, [4](#)
- setDebounceTimeMS
 - SWITCHBUTTON, [7](#)
- setState
 - SWITCHBUTTON, [7](#)
- SHORTPRESSED
 - SWITCHBUTTON, [4](#)
- simple.ino, [9](#)
- sleepmode.ino, [10](#)
- SWITCHBUTTON, [1](#), [3](#)
 - buttonLogicalStates, [3](#)
 - buttonPhysicalStates, [4](#)
 - checkButton, [5](#)
 - DEBOUNCING, [4](#)
 - getButton, [6](#)
 - getState, [6](#)
 - IDLE, [4](#)
 - INPROGRESS, [4](#)
 - LONGPRESSED, [4](#)
 - LONGPRESSEDRELEASED, [4](#)
 - MISSED, [4](#)
 - PRESSED, [4](#)
 - readyForSleep, [7](#)
 - RELEASED, [4](#)
 - setDebounceTimeMS, [7](#)
 - setState, [7](#)
 - SHORTPRESSED, [4](#)
 - SWITCHBUTTON, [4](#)
 - UNKNOWN, [4](#)
- SWITCHBUTTON.h, [10](#), [13](#)
 - DEBOUNCEMS, [12](#)
 - LONGPRESSEDDEADTIMEMS, [12](#)
 - LONGPRESSEDMS, [12](#)
 - SWITCHBUTTON_VERSION, [12](#)
- SWITCHBUTTON_VERSION
 - SWITCHBUTTON.h, [12](#)
- UNKNOWN
 - SWITCHBUTTON, [4](#)